AR GROUP PROJECT · PFH HANSECAMPUS STADE

FR3 / KUKA DIGITAL TWIN DOCUMENTATION

Robot arm digital twin, KRL-style scripting language, and live dual-robot switcher — built from scratch in the browser.



13 / 14
TASKS COMPLETE

1
TASKS REMAINING

2
ROBOTS SUPPORTED

2026

Rohan Pagadakula · July 27, 2026

spidey4343.github.io/Panda-DigiTwin

1. Overview

This project is a browser-based digital twin of a robot arm, built around a small KRL-style programming language: a parser converts KRL text into an intermediate representation (IR), an async interpreter walks that IR, and a backend layer translates each instruction into motion on a live 3D model. The 3D model, inverse/forward kinematics, and a collision/ground safety layer are all implemented from scratch in JavaScript, rendered via the X_ITE X3D engine.

The Franka FR3 was chosen as the primary robot out of personal interest, rather than the KUKA model suggested for the group. Following the suggestion that the application be built so it could adapt to a different robot's geometry, it was later extended into a live dual-robot switcher: the same application now also drives a KUKA KR4R600 model, and a single click in the header swaps the entire app — 3D model, joint sliders, kinematics, and safety checks — between the two robots. The KUKA model and its original inverse-kinematics code were built by a classmate, Sanath, in a previous AR_2026 DigiTwin cohort, and are reused here with credit and adapted to plug into this app's architecture.



Live app — FR3 in its default HOME pose, joint sliders and inverse-kinematics panel on the right.

2. Completed Tasks

13 of the 14 tasks on the AR Digital Twin Planner board are complete. Each has supporting evidence (a short write-up plus the actual implementing code, or generated IR/JSON for PAR-02) delivered separately as task-evidence files.

Task	Description & acceptance criteria	Status
PAR-01	Parse KRL line-by-line (e.g. `LIN {X 100,Y 200,Z 300}`). Each line converted to IR; no crash.	**Complete**
PAR-02	Convert parsed commands to structured IR. JSON-like IR compatible with execution.	**Complete**
EXE-01	Execute IR sequentially. Runs commands correctly; basic error handling.	**Complete**
EXE-02	Map IR types to execution functions. LIN, PTP, WAIT working.	**Complete**
INT-01	Unified backend API for JS/Python/MATLAB: `move_ptp`, `move_lin`, `wait`, `set_output`. Same program runs on 2 backends; no direct backend calls.	**Complete**
INT-02	Backend switching. `KRLEngine.setBackend()` hot-swaps the active backend at runtime; the same parsed KRL program runs correctly against either.	**Complete**
MOT-01	Implement PTP motion (target pose only, not path). Motion visible in twin.	**Complete**
MOT-02	Improve LIN stability and accuracy. Smooth motion; no crash.	**Complete**
CTRL-01	IF / ELSE control flow. Correct branching.	**Complete**
CTRL-02	WHILE loop. Loop exits correctly.	**Complete**
CTRL-03	WAIT FOR (system call, ms parameter).	**Complete**
IO-01	Variable declaration and assignment. INT, BOOL supported.	**Complete**
IO-02	Implement `$IN` and `$OUT` signals. Signals readable and writable.	**Complete**

3. Non-Integrated Modules

One task from the Planner board remains open, plus a smaller gap worth flagging honestly.

INT-03 — Interface to Node-RED

What it's for: exposing the digital twin over MQTT so Node-RED flows could trigger motions or read robot state externally.

Status: not started. Blocked on MQTT broker connection details (host, port, credentials, topic scheme) that haven't been provided/decided yet.

Smaller known gap

- **KUKA KR4R600 safety thresholds** (ground clearance / self-collision distances) are a conservative estimate based on the robot's physical geometry, not pose-by-pose empirically calibrated the way the FR3 profile was (tuned against several known-safe and known-bad test poses). Treat it as a reasonable approximation rather than a certified safety boundary.

4. Architecture

pandaKRL.js — KRL parser (text → AST/IR), IR dispatcher, async interpreter with WHILE/IF support and an infinite-loop guard (5000-iteration cap), and two interchangeable execution backends implementing the same unified 4-call API: the JS backend (drives the live 3D twin) and a Simulated backend (tracks a virtual robot's state numerically, no 3D motion) — a dropdown in the Program tab hot-swaps between them via `KRLEngine.setBackend()`, proving the interpreter never depends on which backend is active.

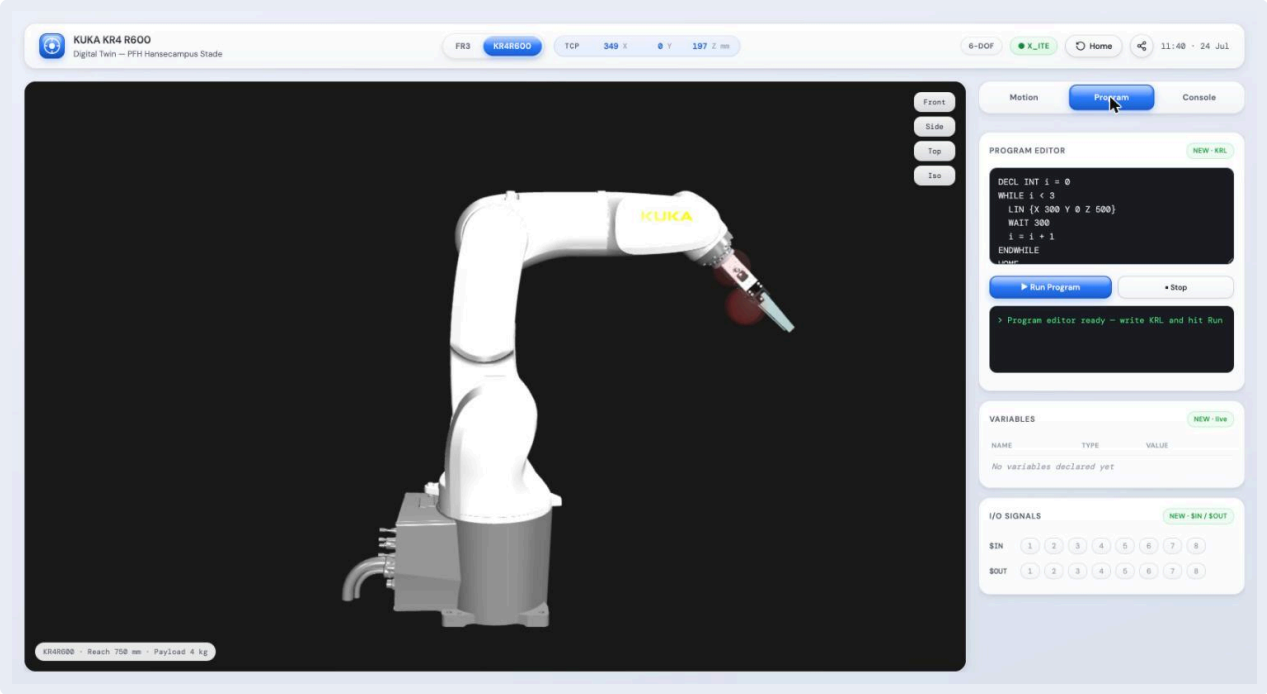
pandaIK.js (FR3) / kukaIK.js (KUKA) — per-robot forward and inverse kinematics. FR3 uses a DH-parameter transform chain with a numerical Jacobian and damped-least-squares IK solver; KUKA uses the same style of DH chain with a Jacobian pseudo-inverse solver seeded by an analytical 3-DOF initial guess. Both expose an identical interface (`getTCP`, `solveIK`, `checkWorkspace`, `getLinkPoints`, `HOME`) so a single IKController can route between them with no per-robot branching anywhere else in the app.

pandaSliders.js — joint-slider UI generalized to any joint count, rotation axis, and range via a per-robot `ROBOTS` config table; drives the live 3D model directly and gates every move through the safety layer before applying it.

pandaSafety.js — capsule/point-approximation collision checking (ground-plane violation + self-collision) per robot, via a `PROFILES` table holding separate calibration constants for FR3 and KUKA.

index.html / style.css — UI shell: the 3D viewport (X_ITE X3D renderer), joint controls, IK panel, KRL program editor/terminal, and the robot switcher.

panda.wrl / kuka.wrl — VRML/X3D scene files describing each robot's geometry, joint hierarchy, and camera viewpoints.



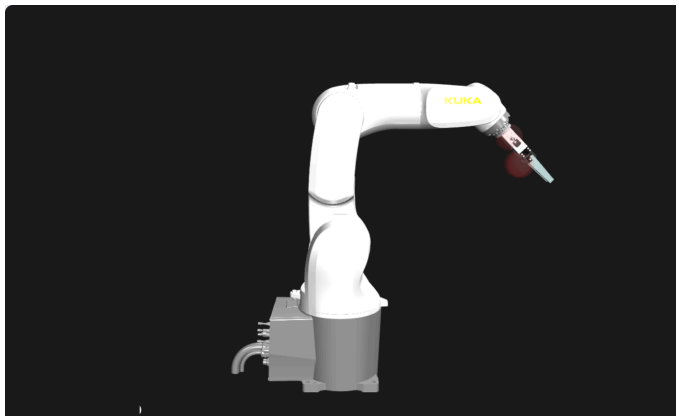
The KRL program editor and console — live, running against the KUKA backend.

5. Dual-Robot Switcher

Clicking FR3 or KR4R600 in the header swaps the entire application — 3D model, joint sliders, kinematics, and safety checks — live, in the same page, with no reload. Both robots share every layer of the app identically: the KRL programming language, the safety system, IK solving, the joint slider UI, and PTP/LIN motion. Nothing in the shared code path is FR3-specific; everything is driven by a per-robot config table. Each robot's HOME pose, workspace limits, and safety thresholds were individually set and verified working live in the browser.



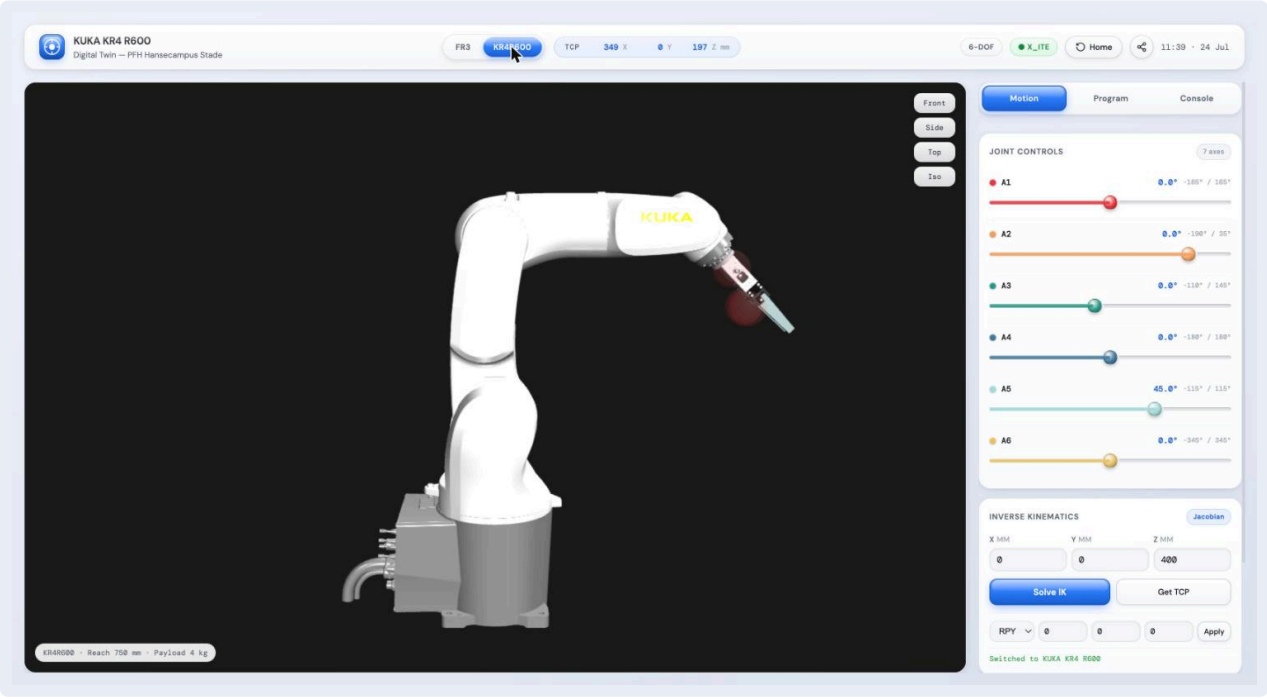
FR3 — 7-DOF, HOME pose



KUKA KR4R600 — 6-DOF, HOME pose

6. How to Use

- **Switching robots:** click FR3 or KR4R600 in the header. The 3D model, joint count/names, and workspace limits update instantly.
- **Joint control:** drag a slider to rotate that joint (in degrees). A move that would cause a self-collision or ground contact is blocked and shows a warning.
- **Inverse kinematics:** enter a target X/Y/Z (mm) and click "Solve IK" — the arm moves to reach that point if it's reachable and safe. "Get TCP" reads the current end-effector position back into the fields.
- **Orientation:** enter a target roll/pitch/yaw (degrees) and click "Apply" — runs a combined position+orientation solve (6-DOF) so the end-effector reaches both the target point and the target orientation. Reports the achieved position and orientation error, and honestly reports "best effort" rather than a false success if a combination is out of reach.
- **View buttons:** Front / Side / Top / Iso switch between fixed camera presets.
- **KRL program editor:** write a small KRL-style program (DECL, WHILE, IF, LIN, PTP, WAIT, $OUT) and click Run. The Console tab shows a live execution log; quick-command buttons insert ready-made templates.
- **Backend selector:** a dropdown above the Program editor switches between the JS backend (drives the live 3D twin) and a Simulated backend (runs the exact same program against a virtual robot state instead — nothing moves on screen, but the log confirms every instruction still executed, demonstrating the interpreter is genuinely backend-agnostic).
- **Share button:** copies a shareable link to the clipboard (confirmed with a toast notification).
- **Home button:** resets the active robot to its HOME pose.



KUKA KR4R600 selected — joint controls (A1–A6) and inverse-kinematics panel.

7. Deployment

The application is deployed and live in two places, both running the same code:

- **PFH server:** https://automation.pfh.de/AR/pagadakulaBR/ (branch `pagadakulaBR`)
- **GitHub Pages:** https://spidey4343.github.io/Panda-DigiTwin/

8. Attribution

The KUKA KR4R600 3D model, its joint-hierarchy PROTOs, and the original inverse-kinematics solver were built by a classmate, Sanath, in a previous AR_2026 DigiTwin cohort. They are reused here with credit and adapted to this app's shared robot-agnostic architecture (interface wrapping, path repointing, viewpoint conventions) — see the header comments in `kuka.wrl` and `kukaIK.js` for the specifics of what was changed.

All screenshots in this document are captured directly from the live application at the URLs above.